



ForgeMind™

Technical Brief

VERSION 1.2.5 · SEPTEMBER 2026

Forged For Speed. Refined by Craft.

SEAGLASS FOUNDRY LLC

*The GIS Intent Compiler. Natural language in, validated geospatial pipelines out
— or an honest refusal.*

Executive Summary

ForgeMind™ is the natural-language control plane for GPU-accelerated geospatial analysis. It takes a plain-language request — "map the slope around Objective Bravo", "where can we land outside the enemy viewshed", "rebuild the elevation picture after the objective displaced" — and turns it into a validated, executed pipeline on ForgeGIS™, or, when the request cannot be honored correctly, into an explicit refusal rather than a confident guess.

It is built on one architectural choice. A typical agent hands the whole problem to a model in a tool-calling loop and hopes the model strings the right calls together. ForgeMind narrows the language model to what it does reliably — understanding what the operator wants — and hands everything that must be correct to deterministic machinery. On that compiled path a request either becomes a type-checked pipeline or is declined with a reason.

Zero

confident-wrong answers
across the June 2026
conversation validation
campaign

~94%

of pairs passing the June 2026
compiler evaluation — intent
to validated pipeline, on
Claude

~25 ×

less model spend than a legacy
agent-loop run on one site-
suitability request (Claude,
May 2026)

5

ways to run it: Claude,
OpenAI, Gemini, Grok, or a
model server you run yourself

Each figure is scoped to the run that produced it; the *Validation* section says exactly what each one rests on and what it does not.

New in 1.2.5

- **Four vendors or your own hardware.** Gemini and Grok join Claude and OpenAI, and OpenAI now runs on the compiled path; a local model on any OpenAI-compatible server plugs into the same client.
- **The window you type into answers.** Each request names its surface — ForgeGIS Studio™ or Seaglass Globe™ — and carries the scene the operator was looking at as structured context.
- **Batch work, brokered.** With the compute broker installed, ForgeMind hands a folder batch of a supported single raster operation to it as one job, spread across GPU workers.
- **Bounded, and harder to misuse.** Every inbound MCP message is size-capped, and tool results entering the rescue loop's prompt are bounded; browser writes from origins other than ForgeMind's own are refused; a set of misconfigurations is refused at boot rather than discovered mid-conversation.
- **Answers narrated from templates.** Most answers are now written from the computed facts with no model call.

What ForgeMind Is

ForgeMind is the orchestration and intent layer between a person (or another agent) and ForgeGIS. It decides what to run, binds the request to real data through ForgeData™, dispatches the work, and narrates the result — turning an

informal request into a correct, reproducible operation.

ForgeMind and ForgeGIS are built for each other. ForgeGIS keeps its raster data resident on the GPU across a multi-step pipeline, so a chain of operations runs at machine speed rather than round-tripping through disk — which is what makes it worth letting an operator ask for a viewshed, a slope mask and a hillshade in one breath. In the Forge suite, ForgeMind pairs ForgeGIS with the ForgeData catalog and with one or both of the suite's map surfaces, ForgeGIS Studio and Seaglass Globe.

The defining commitment inverts the usual agent design. A conventional agent treats the model as a general-purpose planner that authors tool chains directly, so every wrong parameter and hallucinated step lands looking confident. ForgeMind treats the model as an intent classifier and parameter extractor. On the compiled path the structure of the pipeline — which operations run, in what order, with what types — is owned by deterministic code that validates every step before anything executes.

Who it is for. Operators and analysts who want geospatial products in plain language without learning a query syntax or a desktop GIS; and system builders who want to expose geospatial analysis to AI agents and automation, deterministic where it matters and honest when it cannot proceed.

Architecture

The internal mechanism — the GIS Intent Compiler — is proprietary; what follows is the externally observable contract.

The model does only intent

On the compiled path the model classifies what the operator is asking for and extracts its parameters — an area, an observer point, a slope threshold, a radius. It does not author the pipeline. For a templated capability, the same parsed intent produces the byte-identical pipeline every time — no hallucinated parameters in the pipeline, no operation-arity slips. The intent parse itself is a model call, so borderline requests can parse differently from run to run. A request no template covers is adapted from a reference workflow by the more capable model tier, and then validated like everything else.

Validation before execution

Every candidate pipeline is checked against a type system and operational guards before any work is dispatched. A request with an ambiguous reference, or a place name that does not match its coordinates, is always put back to the operator as a question. A request that is simply below the confidence floor, or that no template or reference workflow can express, is declined by the compiler and may be passed to the guarded agent loop below. A structural failure — data, type check, execution — or an out-of-scope request is refused outright with a specific, machine-readable reason.

Where the guarantee stops. When the compiler declines a request as below its confidence floor or unroutable, ForgeMind falls back to a guarded agent loop in which the model chooses the tool calls. That rescue path keeps unusual requests answerable, and it is why "refusal by construction" is a property of the compiled path rather than of every turn. The loop is capped, cannot change the data catalog, and its tool list can be narrowed by configuration; the suite leaves it at every wired tool, and the loop cannot be switched off.

The right model for each job

With a hosted vendor, ForgeMind runs a fast, low-cost model for intent and a more capable one only for requests no template covers. Since 1.2.5 the final narration is written from templates by default, calling the fast model only when a fact is not already text. On one site-suitability request on Claude in May 2026, against a legacy agent-loop run that hit its iteration cap without finishing, the compiled path cost roughly a twenty-fifth as much; that is a single measured request, not a general ratio. In the suite a local model runs every stage on the one model you choose, and costs hardware rather than tokens.

Four vendors, or your own model

The provider and model sit behind one client abstraction: **Claude, OpenAI, Gemini and Grok** are built in, each with a fast and a strong model, and a model you host on any **OpenAI-compatible server** — Ollama is the tested path — plugs into the same point with `FORGEMIND_LLM_BASE_URL`. Switching provider is configuration plus a restart, not code. The compiler's prompts were tuned on Claude; parse quality on other vendors is something to measure on your own workload, and the non-Claude model identifiers and prices come from vendor documentation.

Native MCP, both directions

As a client, ForgeMind spawns and supervises its MCP backends — ForgeGIS, ForgeData, the map surfaces' bridges and, optionally, the compute broker — as stdio children, with a circuit breaker per namespace, respawn supervision, and a distinction between a slow call (which fails alone) and a hung worker (which is recycled). As a server, it exposes a single high-level tool a higher-order agent can hand a whole geospatial workflow to, plus a structured surface — `list_capabilities` and `run_capability` — that runs a capability with no model call at all.

Named scene entities and the scene

Named geometry an operator places on a map — a dock, a route, an objective — is referenced by a stable handle and hydrated from the catalog when an operation needs it, never re-derived from a bounding box. The design is fail-closed: if the true geometry is not reachable, the operation defers and asks. New in 1.2.5, each request can carry the scene as structured context — entities with their symbol codes, draped overlays, and a scenario clock the answer can use.

The surface per request

Each request names the surface that asked — `studio` or `seaglass` — and the answer is drawn there; an unknown surface is refused. The Control Center's own Ask tab uses a default surface. A deployment with no map surface wired runs

headless and returns answers as text.

Capabilities

ForgeMind makes the ForgeGIS catalog reachable in natural language, anchored on a deterministic core of routed capabilities and extended across the long tail by adapting reference workflows. Coverage grows capability by capability as each is validated; ForgeMind advertises only what it can run. The capabilities it lists can also be called directly, without a model, through `run_capability`.

Conversation, not one-shots. ForgeMind holds context across a session: establish a subject, then refine it — change a parameter, mask a result, re-run with a new value, reference an earlier product. It resolves references like "re-run that with a wider radius" or "mask it to slopes under seven degrees" and named anchors like "Objective Bravo" against a ledger of what the session has established. On the capabilities that take one, an option the operator states — a curvature type, an azimuth, a sight range, an output unit such as feet or nautical miles — reaches the computation. A reference that cannot be resolved is refused with a reason, not guessed.

Representative requests:

- *Direct analysis* — "Compute viewshed from 34.5N 69.2E with a 10 km radius."
- *Discovery into analysis* — "What elevation datasets do we have for this region? Pick the highest-resolution one and run a slope analysis."
- *Site suitability* — "Where can we land outside the enemy viewshed?"
- *Scene-aware refinement* — "Mark Objective Bravo here and map the slope around it", then later "rebuild the slope picture — it displaced to a new grid reference."
- *Batch* (1.2.5) — "Hillshade every tile in this folder", submitted to the compute broker as one job.

Honest answers. Under-specified, out-of-scope or unbindable requests get a clarifying question, a statement that no data covers the area, or a structured refusal. A slow or failed backend fails its capability cleanly while the rest keeps serving. Contour, buffer and route results too large to inline are published from the engine's file when it is in longitude and latitude, and refused — never drawn as degrees — when it is not.

Validation: What Each Figure Rests On

Figure	What it rests on	What it does not show
Zero confident-wrong answers	The June 2026 multi-turn conversation campaign — thousands of turns over dozens of sessions, run before the 1.2.5 provider options existed	A property of every future turn; the changelogs record confident-wrong defects found and fixed in validation since June
~94% compiler pass rate	118 of 126 intent-to-pipeline pairs on the June 2026 evaluation set, on Claude, including continuation, overrides, named anchors and must-refuse cases	A current rate: the evaluation set has more than doubled since June and no full run on it is recorded; no pass rate exists for other vendors
~25× lower model spend	One site-suitability request, May 2026, on Claude, against a legacy agent-loop run that hit its iteration cap without finishing — a request chosen because the loop struggled with it	A general ratio across workloads, or a figure for other vendors or local models

The marathon showcase. The published S-MARATHON-22 session — 102 operations planning a scripted search-and-rescue scenario against real SRTM terrain, all succeeding — is a regression run from ForgeMind's test harness. The harness supplies each turn's intent from the scenario script, and the map client is a stand-in, so the session exercises everything after the model: routing, type-checking, resolving named entities across a hundred turns, and executing real analysis. It says nothing about model cost or latency.

What is and is not guaranteed. ForgeMind's correctness boundary is its own: intent resolution, validation, refusal discipline, dispatch, and its HTTP and MCP surfaces. The correctness of the geospatial computations belongs to ForgeGIS, validated separately against its own oracles, and is reproduced faithfully because ForgeMind calls the same implementations whether a person or an agent asks.

Deployment and Integration

ForgeMind is one self-contained Java application, run in either of two shapes from the same artifact:

- **HTTP service** — a buffered request endpoint and a streaming (Server-Sent Events) endpoint that surfaces progress as it happens, plus health and capability endpoints and an opt-in Prometheus metrics endpoint. It binds `127.0.0.1` by default. A bearer token, a rate limit, a request-size cap and a concurrency cap are built in; in the Forge suite the Control Center sets a per-install token. New in 1.2.5, a browser write from any origin other than ForgeMind's own loopback address, unless explicitly allowed, is refused, closing a cross-site and DNS-rebinding hole without affecting native clients.
- **MCP stdio server** — ForgeMind as a delegatable tool to a higher-level agent host, with the zero-model capability surface beside it.

In the Forge suite the Seaglass Control Center™ installs ForgeMind with a private Java runtime, keeps provider keys in the Windows Credential Manager and hands only the active provider's key to ForgeMind, starts it on port 8080 after

the map surfaces, and wires Studio's bridge, the Globe and, if installed, the compute broker. Switching provider is a choice in Settings and a restart.

Refused at boot, not mid-conversation. An unknown provider, two map surfaces with no default, an unresolvable bind address or a port already in use, a malformed backend command, and a ForgeGIS older than the contract ForgeMind requires all stop ForgeMind at startup with a reason.

Disconnected sites. With a model server on the site's own network, no request goes to a cloud model vendor. ForgeMind itself needs only local disk; ForgeData fetches public data sources when asked, so a disconnected site stages its data in advance.

Known Limits in 1.2.5

- The provider is read at startup; switching needs a restart.
- A local model is slower than a hosted one — each request may take minutes — and in the suite runs every stage on one model; Ollama must be given enough context, or it truncates silently.
- OpenAI, Gemini and Grok on the compiled path were exercised against a test stub, not live vendor keys, and their model ids come from vendor documentation; Ollama is the only local server tested.
- The HTTP surface has no TLS; the rate limit is off by default; there is no multi-tenant isolation and no audit log.
- The broker batch lane runs one of a set of single-input raster operations, with default parameters, over a folder; multi-operation batches are follow-on work.
- A request outside the templated capabilities goes to the adapting model or the rescue loop, and may be refused.

Specifications at a Glance

Role	Natural-language control plane for ForgeGIS — the GIS Intent Compiler
Shapes	HTTP (buffered and SSE streaming) · MCP stdio server with a zero-model capability surface
Providers	Claude · OpenAI · Gemini · Grok · any OpenAI-compatible server (Ollama tested); one active at a time
Backends	ForgeGIS and ForgeData as MCP stdio children; Studio bridge and Seaglass Globe surfaces; optional compute broker
Surfaces	Per request — Studio or Seaglass Globe; a default for the Control Center's Ask tab; headless supported
Safety	Validation before execution; named refusals; per-namespace circuit breakers; capped MCP messages and bounded rescue-loop results; cross-origin write guard; boot-time refusals

In the suite	Port 8080, loopback, per-install bearer token; keys in the Windows Credential Manager
Runtime	One Java application; the suite brings a private Temurin JRE 25
License	Proprietary. Part of the commercially licensed Forge suite; evaluation builds available to qualified teams

Licensing and Contact

ForgeMind is proprietary software of Seaglass Foundry LLC, distributed as part of the Forge suite through the Seaglass Control Center. The Forge suite is commercially licensed; terms are provided on request, and evaluation builds are available to qualified teams. Nothing in this brief is a grant of rights.

Contact. rich@seaglassfoundry.com · seaglassfoundry.com

ForgeMind™, ForgeGIS™, ForgeData™, ForgeGIS Studio™, Seaglass Globe™, Seaglass Control Center™, and Seaglass Foundry™ are trademarks of Seaglass Foundry LLC. Claude is a trademark of Anthropic, PBC; OpenAI and GPT are trademarks of OpenAI; Gemini is a trademark of Google LLC; Grok is a trademark of xAI; Ollama is a trademark of its respective owner. The Model Context Protocol, Java, Temurin, SRTM, and other marks referenced are the property of their respective owners; reference to them does not imply endorsement. © 2026 Seaglass Foundry LLC. All rights reserved.